Arnaud ALET & Andrej Zlatevski - MoSIG - UGA
github : https://github.com/arnauda-gh/Distributed-Memory-IDS

Distributed Shared Memory Project - ZLATEVSKI Andrej and ALET Arnaud - 2026

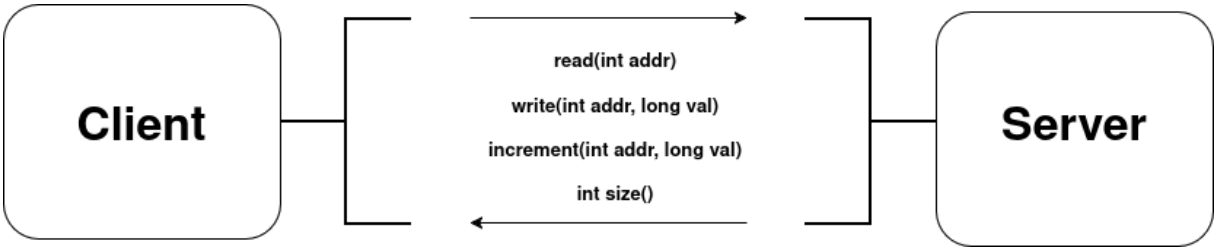
Summary

This is a distributed shared memory model, assuming bidirectional communication, failure transparency and reliable network on a clique topology between compute nodes. We use a replicated memory approach, with Ricart and Agrawala's mutual exlusion algorithm on a clique topology. Java RMI was used to define Client <=> Controller interaction (within node) and RabbitMQ was used as a broker service to define the uncoupled communication between compute nodes. The consistency model chosen for this project is strong consistency, putting a strong accent in respecting the global order of requests/events coming from the nodes.

Architecture

Before defining the architecture, we will describe the assumptions. As mentioned in the summary section, we assume bidirectional communication, which implies a clique topology is in place. Then, we assume failure transparency, meaning neither of the entities crash during exection, such as the RabbitMQ broker and the compute nodes. And finally, we assume reliable network, making sure the messages always arrive in the destination compute node network buffers or in the RabbitMQ broker queues. From the point of view of the user, no restrictive assumptions were defined, because the interface is simple. We prioritized correctness over quantity of features in the implementation, given the limited time.

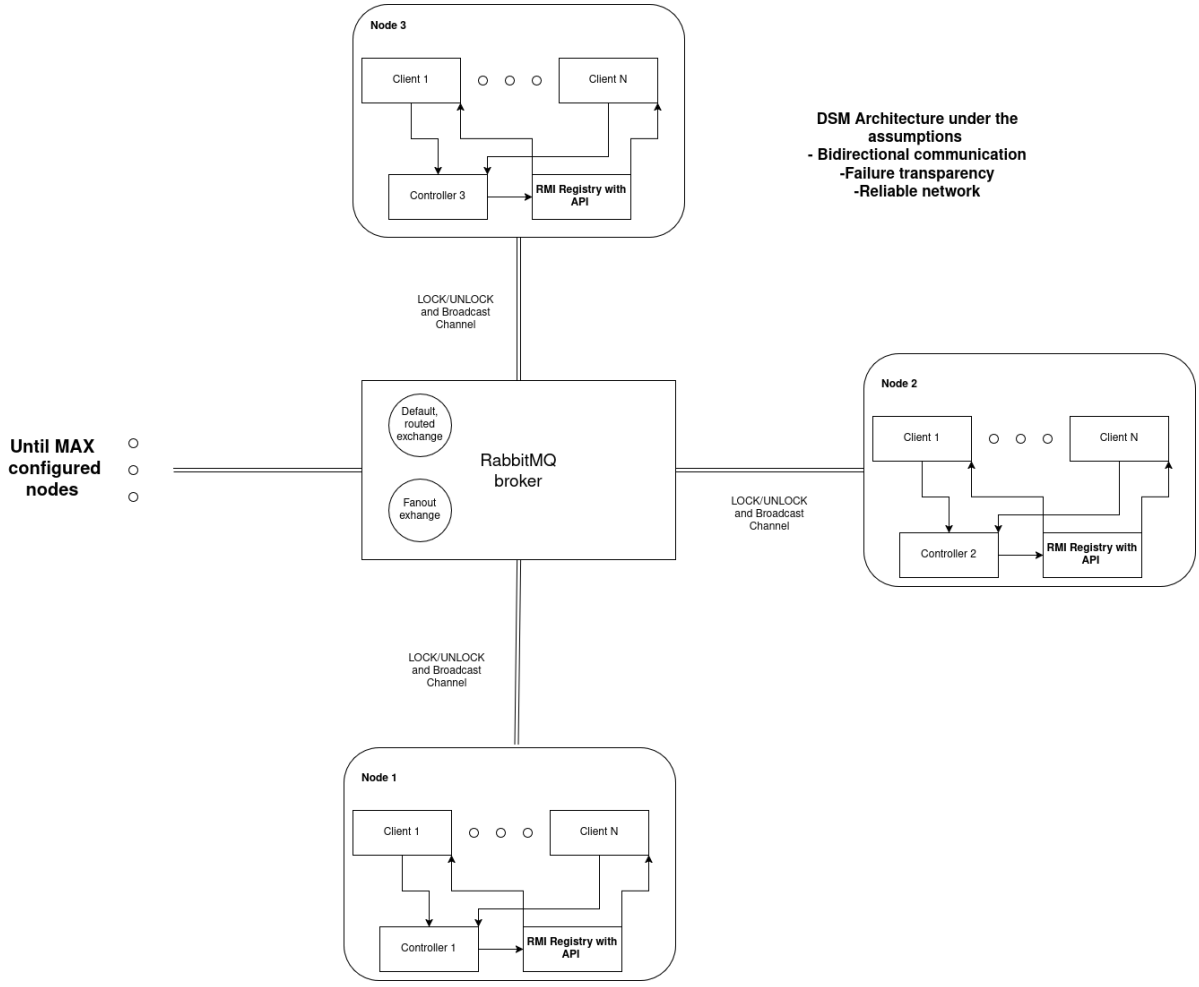
We thus introduce the interface and the interaction between a local client and controller (server), both of them living in the same node and using Java RMI:



File : RMI interface BETTER.drawio

We chose this approach because calling the remote procedures via RMI are synchronous, abstracting process suspension and resumption. The controller deals with the client's request(s), and sends back the result(s) to the client, using the proxy design pattern, as seen in class. Presented in the figure above, we expose 5 functions to the client, with the memory model being as simplistic as possible: An array of long values, with the indices being the addresses.

We now proceed to the big picture, the general architectural idea (for instance if 3 nodes are interacting): NOTE: Zooming is encouraged if not visible.



File : DSM-FINALFINAL.drawio

We first explain the design choices within a node. In each node, there is a controller that deals with remote call procedures from local clients, just as well the communication between the nodes via the RabbitMQ broker. The controller exports the interface object to the RMI registry living inside the node, so that the local clients can locate it and use its functions. Because we deal with RMI, the controller is the one that has access to the memory object, which means this memory is local to ALL clients within the node. The details of synchronization between local clients is explained in the next section.

We proceed with the external communication between the nodes. For each node, there is an exclusive queue for both LOCK/UNLOCK and broadcast protocols. The reason for this choice is because both types of interactions do not overlap, so 1 queue is enough. However, this expresses the need for 2 exchanges in the RabbitMQ broker, which are the default and fanout (broadcast) exchanges. Binding the queues to the broadcast exchange is necessary, so that the communication routes are open between the nodes.

Implementation

We define 3 phases from the moment a node connects to a broker:

- Handshake: This is essential, in order to identify all nodes connected in the clique configured network. The exchange used for it is the fanout.

- Mutual Exclusion: To maintain strict consistency across nodes, the implementation employs a distributed mutual exclusion algorithm inspired by Ricart-Agrawala. Before executing any write operation, a controller must acquire a global lock by exchanging timestamped requests and resolving priorities based on logical clocks. The exchange used is the default one.

- Global memory update: Once the critical section is entered, the controller updates its local state and broadcasts a GLOBAL_UPDATE message to all peers. The operation is blocked until all nodes acknowledge the update, ensuring that all copies of the memory are coherent before the lock is released and subsequent operations are allowed. The fanout exchange is appropriate for this case.

One obvious limitation in our system to point out is our vision of which regions of the memory we lock/unlock. Because we represent the memory with an array, we could either set the granularity of the way we are handling the mutual exclusion either by address (meaning if 2 different nodes write to distinct addresses, we should

authorize it) or by memory (meaning we limit to only 1 write operation per time globally, no matter the address considered). Here, because of time, and possibly performance, we chose the second option.

Internally, the Controller coordinates three specialized components: the Memory class for local data storage, the MutualExclusionHandler for RabbitMQ-based synchronization, and the RMIHandler for client access. The Memory class uses local synchronization primitives to manage concurrent access within the node, while a simulated latency is introduced to validate the robustness of the distributed locking mechanism. This layered structure, combined with a robust PacketDispatcher for asynchronous messaging, ensures that the complex state transitions of the distributed protocol are handled reliably across the network.

The project is organized into three modules to ensure a clean separation of concerns: Lib encapsulates the shared RMI interfaces and utilities required by both ends, Controller implements the core distributed memory logic and synchronization protocols, and Client provides an independent interface for users to execute operations on the distributed system. This structure allows the shared API to be maintained in a single location while keeping the complex coordination logic of the nodes isolated from the client-side implementation.

Deployment

Compilation & Installation

The project uses Gradle for build automation and dependency management. The project is separated into 3 sub-project :

- `Controller` : contain the code for a running node (manage synchronisation of the memory, and client handling).
- `Client` : contain some default client for test and show the behaviour of our implementation.
- `lib` : contain code shared between the `Controller` and the `Client` sub-projects (mainly `ControllerAccess`).

To compile all modules and download necessary dependencies (RabbitMQ client, SLF4J), run the following command from the root directory:

- `./gradlew build`

Execution

The system requires a running RabbitMQ broker which is used in communication between Controllers. By default, the system attempts to connect to a broker on `localhost`.

1. **Start a Controller:** Each node in the distributed system must run exactly one Controller instance.
   - `./gradlew :Controller:run --args="<rabbitmq-host>"`

   Alternatively, using the Makefile:
   - `make controller`

   Upon initialization, the Controller will output the dynamically allocated RMI registry port (`RMI registry started on port: 12345`). This port while be used in the client start process.
2. **Start a Client:** Clients connect to a local Controller via Java RMI to perform memory operations.
   - `./gradlew :Client:run`

   Alternatively, using the Makefile:
   - `make client`

   The Client will prompt for the RMI port displayed by the Controller.

Conclusion

State and improvements

Our implementation successfully delivers a replicated distributed shared memory system that guarantees strong consistency. We opted for a simplified “global lock” approach where any write operation locks the entire memory array. While this ensures strict ordering and correctness, it is a “minimal” implementation in terms of concurrency. A more advanced version would implement page-based or address-based granularity to allow simultaneous writes to disjoint memory regions.

Choice of Protocols

The combination of Java RMI and RabbitMQ proved to be effective. RMI abstracted the complexity of local process communication, while RabbitMQ provided the necessary uncoupling for a robust distributed back-end. Using the `lib` sub-project to share the status of the client interface, was really easy to use (adding, removing some features).

Final Thoughts

This project was really interesting, as the subject was a personnal choice. Technologies that we used were new but pretty intuitive. Our work as a group was really enjoyable, good communication and great dispatching of the work.

Arnaud ALET & Andrej Zlatevski - MoSIG - UGA
github : https://github.com/arnauda-gh/Distributed-Memory-IDS